

# ShortStop

Counterexample-Guided Short-Horizon Shielding of Generative Robot Policies

---

Cong Tu

Ngày 21 tháng 8 năm 2026

# Overview

- Part 0 — From Classical Control to Generative Policies
- Part 1 — Problem Formulation & Core Insight
- Part 2 — Existing Safety Machinery & Gap
- Part 3 — SHORTSTOP Runtime Shield
- Part 4 — The P-R-C-S Framework
- Part 5 — Experimental Setup
- Part 6 — Guarantees & Challenges
- Track A & Track B — Kế hoạch song song (lý thuyết + thực nghiệm)
- Tổng kết

## Part 0 — From Classical Control to Generative Policies

---

## Bài toán tổng quát: robot cần quyết định gì, ở đâu?

- **Bối cảnh thực tế:** robot tay máy (robot arm) trong nhà máy/kho hàng/nhà bếp — ví dụ gấp vật, tránh chướng ngại vật, đặt đồ đúng vị trí.
- **Input:** quan sát hiện tại  $o_t$  — ảnh camera, vị trí/vận tốc khớp, vị trí vật cản hoặc mục tiêu.
- **Output:** hành động  $a_t$  — lực/vận tốc từng khớp, hoặc vị trí đích của tay gấp.
- **Bài toán chung:** cần một hàm quyết định  $a_t = \pi(o_t)$  sao cho robot *hoàn thành nhiệm vụ và không gây nguy hiểm* (không đâm vào người/vật, không vượt giới hạn khớp).

### Câu hỏi xuyên suốt lịch sử ngành

Hàm quyết định  $\pi$  này được **xây dựng như thế nào?** — câu trả lời đã thay đổi qua nhiều giai đoạn (slide tiếp theo), và ShortStop nằm ở giai đoạn mới nhất.

## Từ Classical Control đến Generative Policies

- **Control cổ điển** (PID, LQR): luật điều khiển suy từ phương trình vật lý  $\Rightarrow$  **chứng minh được** an toàn/ổn định.
- **MPC**: giải bài toán tối ưu có ràng buộc mỗi bước (receding horizon)  $\Rightarrow$  ràng buộc nằm ngay trong QP.
- **RL / imitation learning**: policy học từ dữ liệu  $\Rightarrow$  **mất khả năng chứng minh** — policy là hộp đen.
- **Safe RL / CBF-QP**: cố lấy lại bảo đảm bằng barrier function, nhưng chỉ sửa *một* hành động.
- **Generative policy (diffusion/flow)**: sinh cả *chuỗi* hành động, không có certificate nào.

# Model Predictive Control (MPC)

- **Ý tưởng:** tại mỗi bước thời gian, giải một bài toán tối ưu có ràng buộc, dự đoán trước  $N$  bước tương lai bằng mô hình động lực  $f$ , tìm ra cả chuỗi hành động tối ưu — nhưng chỉ **thực thi hành động đầu tiên**, rồi giải lại từ đầu ở bước kế tiếp (*receding horizon*).
- **Vì sao vẫn chứng minh được an toàn:** ràng buộc vật lý (giới hạn khớp, tránh vật cản) được đưa thẳng vào bài toán tối ưu dưới dạng *constraint* — nghiệm mà bộ giải (QP) trả về, nếu tồn tại, **luôn** thoả ràng buộc đó, không phải "hy vọng" mà là bảo đảm từ chính cấu trúc bài toán.
- MPC là đại diện tiêu biểu của nhóm phương pháp tối ưu có ràng buộc, đồng thời là nền tảng cho nhiều predictive safety filter hiện đại.
- **Giới hạn:** cần biết mô hình động lực  $f$  khá chính xác, và bài toán tối ưu phải giải được *real-time* mỗi bước.

# Reinforcement Learning & Imitation Learning

- Thay vì viết công thức tay, để một mạng nơ-ron  $\pi_\theta$  **học** luật điều khiển từ dữ liệu:
  - *Reinforcement Learning (RL)*: robot tự thử-sai trong môi trường/simulator, nhận thưởng khi làm đúng, phạt khi sai — dần học ra chính sách tối ưu hoá thưởng.
  - *Imitation Learning*: cho robot xem hàng ngàn demo con người điều khiển, học *bắt chước* trực tiếp, không cần reward.
- **Cái mất**:  $\pi_\theta$  giờ là một mạng nơ-ron hàng triệu tham số — không còn phương trình tường minh để **chứng minh** nó luôn an toàn. Chỉ có thể *kiểm định bằng thực nghiệm* (chạy thử nhiều lần xem có sai không), không phải bảo đảm toán học.
- **Cái được**: tổng quát hoá tốt hơn cho tác vụ khó viết tay (ví dụ cầm nắm vật mềm, thao tác khéo léo).

## Safe RL via Control Barrier Functions (CBF-QP)

- **Ý tưởng:** giữ nguyên policy học được (RL) làm việc chính, nhưng thêm một lớp “gác cổng” (safety filter) đứng giữa policy và robot thật.
- **Control Barrier Function (CBF):** định nghĩa một hàm  $h(x)$  sao cho “an toàn”  $\Leftrightarrow h(x) \geq 0$ . Trước khi thực thi, giải một bài toán tối ưu nhỏ (QP) để sửa action đề xuất — **sửa càng ít càng tốt** so với đề xuất gốc, miễn giữ được  $h(x) \geq 0$ .
- **Giới hạn:** (a)  $h(x)$  phải *thiết kế tay* cho từng tình huống nguy hiểm cụ thể; (b) can thiệp ở mức action tức thời, thay vì đánh giá/chứng nhận cả một action chunk..
- CBF-Shield là một baseline quan trọng vì cùng theo triết lý sử dụng lớp safety filter đặt ngoài policy.



# Generative Policies: Diffusion và Flow Matching

- Vẫn là **learned policy**, nhưng thay vì học một ánh xạ trực tiếp

$$o_t \rightarrow a_t,$$

generative policy học một **phân phối trên các hành vi hợp lý** và rút mẫu từ phân phối đó.

- Trong robot manipulation, policy thường sinh một **action chunk**

$$a_{t:t+H} = (a_t, \dots, a_{t+H-1}),$$

giúp duy trì temporal consistency và tăng throughput.

- Diffusion và flow matching là hai cơ chế phổ biến để thực hiện quá trình sinh mẫu này.
- Đây là bước tiến lớn về **multimodality** và khả năng xử lý các thao tác phức tạp mà khó viết tay.

## Nhưng một vấn đề mới xuất hiện

Policy không còn đơn giản là “tính ra một action” — nó **sinh ra một chunk từ một phân phối**, và chunk đó không đi kèm safety certificate.

# Vì sao sinh cả một phân phối thay vì hồi quy một action?

- Với một quan sát, có thể tồn tại **nhiều hành vi đều đúng** — đây là bài toán **multimodality**.
- Ví dụ: robot có thể đi vòng vật cản bên trái hoặc bên phải. Nếu ép policy hồi quy một action duy nhất, các mode có thể bị **trung bình hoá** thành một hành vi không hợp lý.
- Generative policy thay vào đó học

$$p_{\theta}(a_{t:t+H} \mid o_t),$$

rồi **rút mẫu** một hành vi cụ thể từ phân phối này.

- Vì vậy, cùng một quan sát có thể sinh ra nhiều chunk khác nhau — miễn chúng đều nằm trong các mode mà policy đã học.

Regression → một dự đoán duy nhất

Generative policy → phân phối các hành vi khả dĩ

# Generative policy sinh action chunk bằng cách nào?

## Diffusion Policy

noise  $\rightarrow$  denoising steps  $\rightarrow a_{t:t+H}$

- Denoiser được điều kiện hoá theo quan sát  $o_t$
- Biến đổi noise thành action chunk hợp lệ qua nhiều bước lặp

## Flow Matching

noise  $\rightarrow$  ODE flow  $\rightarrow a_{t:t+H}$

- Học một trường vector liên tục
- Việc rút mẫu đi theo ODE đã học được

## Trình tượng hoá chung cho SHORTSTOP

SHORTSTOP không quan tâm chunk được sinh ra bằng cách nào.

$$\pi_{\theta}(o_t) \rightarrow a_{t:t+H}$$

Generative policy được xem như một **black-box chunk proposer**.

## Part 1 — Problem Formulation & Core Insight

---

# Generative policies: năng lực đi kèm khoảng trống an toàn

## Được gì

- Multimodality
- Nhất quán theo thời gian (temporal consistency)
- Thực thi chunk với throughput cao
- Xử lý thao tác phức tạp mà không cần luật điều khiển viết tay

## Mất gì

- Sampler sâu  $\Rightarrow$  hành vi black-box
- Lấy mẫu ngẫu nhiên  $\Rightarrow$  có thể sinh ra chunk khác nhau
- Một chunk trải dài qua nhiều hành động tương lai
- **Chunk sinh ra không đi kèm safety certificate**

Generative policy giải quyết “hành động thế nào”



**Nhưng ai bảo đảm hành vi được đề xuất là an toàn?**

# Vì sao không formal verify trực tiếp generative policy?

## Giải pháp hiển nhiên

Verify trực tiếp policy:

$\forall o, \quad \pi_\theta(o)$  luôn sinh hành vi an toàn.

- Nhưng  $\pi_\theta$  là một deep generative sampler với hàng triệu tham số và nhiều bước nội tại.
- Sau khi sinh chunk, nó còn tương tác với động lực học robot, va chạm và nhiễu từ môi trường.
- Các công cụ neural-network reachability chỉ scale được tới mạng tương đối nhỏ và horizon ngắn.
- Full formal verification của generative policy vì vậy là **không thực tế**.

## Insight cốt lõi của SHORTSTOP

Không cần chứng minh

“policy luôn an toàn.”

Chỉ cần chứng minh

“chunk sắp được thực thi an toàn trong short horizon.”

## **Part 2 — Existing Safety Machinery & Gap**

---

## Các cơ chế an toàn hiện có cho policy đã học

- Khi policy đã được học, một hướng tự nhiên là đặt thêm **safety mechanism** bên ngoài policy thay vì thay đổi controller chính.
- Các hướng gần với bài toán của chúng ta gồm:
  1. **Generator modification**: bias hoặc retrain generative policy để ưu tiên các trajectory an toàn/khả thi.
  2. **MPC-based filtering**: dự đoán tương lai và sửa action bằng constrained optimization.
  3. **CBF-based shielding**: dùng barrier condition để ngăn policy đưa hệ ra khỏi vùng an toàn.
  4. **STL monitoring**: đánh giá trajectory dựa trên một temporal specification.

### Nhưng generative policy đặt ra một yêu cầu mới

Ta không chỉ muốn “sửa action cho an toàn”, mà muốn **đánh giá và chứng nhận một action chunk** trước khi nó được thực thi.



# Gap 1 — Sửa generator hay bảo vệ nó post-hoc?

## Safe generation

training / sampling  $\rightarrow$  policy an toàn hơn

- Safety được đưa vào quá trình training hoặc generation.
- Có thể bias policy về phía feasible/safe behaviors.

## SHORTSTOP

frozen  $\pi_\theta \rightarrow$  safety filter

- Không retrain policy.
- Không sửa denoiser / flow.
- Có thể áp dụng cho policy đã được train sẵn.

### Gap

Các phương pháp safe generation thay đổi chính policy mà chúng ta muốn bảo vệ. SHORTSTOP đặt safety **post-hoc**, bên ngoài một generative policy đã đóng băng.

## Gap 2 — Sửa ở mức action hay an toàn ở mức chunk?

### MPC / CBF filtering

$$\pi_{\theta}(o_t) \rightarrow \boxed{a_t} \rightarrow \text{filter}$$

- MPC có thể dự đoán tương lai nhưng cuối cùng điều chỉnh action được thực thi.
- CBF-QP áp đặt safety condition tại thời điểm hiện tại.

### Generative policy

$$\pi_{\theta}(o_t) \rightarrow \boxed{a_{t:t+H}}$$

- Một lần sampling tạo ra cả action chunk.
- Chunk có temporal coupling: các action trong chunk không độc lập.

### Gap

Safety mechanism cần xét tới **cả short-horizon chunk**, không chỉ một action tức thời.

## Gap 3 — An toàn thiết kế tay hay learned dynamics?

### CBF / classical safety control

$$h(x) \geq 0$$

- Cần một barrier function phù hợp với unsafe set.
- Thường cần mô hình dynamics có cấu trúc rõ ràng.
- Việc thiết kế  $h(x)$  trở nên khó khi môi trường và task phức tạp.

### SHORTSTOP

$$\hat{f}(x, a) \quad + \quad \epsilon$$

- Sử dụng local dynamics hoặc learned transition model.
- Sai số model được bao phủ bởi certified error bound.
- Không yêu cầu một CBF riêng cho từng obstacle/task.

### Gap

Thay vì encode từng vùng nguy hiểm thành một barrier function, SHORTSTOP suy luận trực tiếp trên **reachable state tube** của candidate chunk.

## Gap 4 — Giám sát nominal hay chứng nhận sound trên tube?

### Nominal STL monitor

$$x_0 \rightarrow \hat{x}_1 \rightarrow \cdots \rightarrow \hat{x}_H$$

- Roll out một trajectory duy nhất.
- Tính STL robustness trên trajectory đó.
- Negative robustness  $\Rightarrow$  reject.

### SHORTSTOP

$$x_0 \rightarrow \boxed{\hat{\mathcal{R}}_{0:H}}$$

- Propagate một reachable-set over-approximation.
- STL robustness được lower-bound trên **toàn tube**.
- Sau đó tìm counterexample trong tube.

### Gap

Một nominal rollout chỉ trả lời “trajectory này có vẻ an toàn”. SHORTSTOP cần trả lời mạnh hơn:

mọi trajectory được tube bao phủ đều thỏa safety specification.

# Tổng hợp khoảng trống

| Hướng tiếp cận hiện có                   | Còn thiếu gì                                                 |
|------------------------------------------|--------------------------------------------------------------|
| Generator modification                   | Bảo vệ post-hoc cho một policy đã đóng băng                  |
| MPC / CBF                                | Suy luận an toàn trên chunk trải dài theo thời gian          |
| CBF thiết kế tay / model optimal-control | Suy luận short-horizon tổng quát với learned dynamics        |
| Nominal STL monitor                      | Chứng nhận sound trên reachable-tube + counterexample search |

## Khoảng trống nghiên cứu

Cái còn thiếu là một runtime shield đồng thời cung cấp

post-hoc + chunk-level + tube-based + counterexample-guided

cho một **generative robot policy không bị chỉnh sửa**.

## Part 3 — SHORTSTOP Runtime Shield

---

# SHORTSTOP: lớp an toàn runtime còn thiếu

## Ý tưởng cốt lõi

SHORTSTOP bọc quanh một **generative policy đã đóng băng** và kiểm tra hậu quả ngắn hạn của action chunk được đề xuất trước khi thực thi.



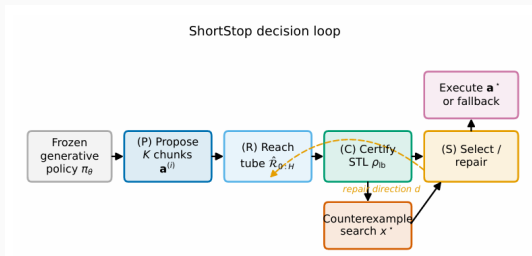
- Policy vẫn chịu trách nhiệm về **task performance / multimodal behavior**.
- SHORTSTOP chịu trách nhiệm về **runtime safety**.
- Policy không cần biết safety mechanism tồn tại ở phía sau.
- SHORTSTOP chỉ can thiệp khi candidate chunk không đạt certified safety condition.

## Một runtime shield cần giải quyết những gì?

- Đứng giữa  $\pi_\theta$  (black-box proposer) và robot thật, một shield ngắn hạn phải trả lời được 3 câu hỏi trước khi cho chunk đi qua:
  1. **Dự đoán (Predict)**: nếu thực thi chunk này, robot sẽ ở đâu trong  $H$  bước tới?
  2. **Đánh giá (Certify)**: quỹ đạo dự đoán đó có vi phạm vùng nguy hiểm  $X_u$  hay không?
  3. **Sửa (Repair)**: nếu vi phạm, có cách nào chỉnh chunk ít nhất để tránh mà vẫn giữ tiến độ nhiệm vụ?
- Ba yêu cầu này dẫn trực tiếp tới kiến trúc của ShortStop và các công cụ lý thuyết mà hệ thống sử dụng.

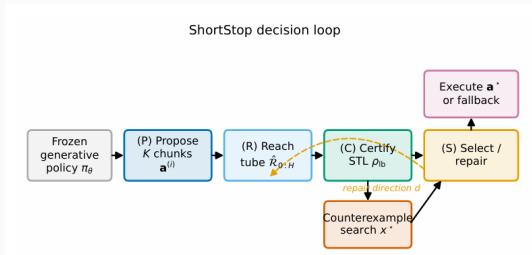


# Các khối kiến thức có sẵn → SHORTSTOP (1/2)



- **Predict** → **Reachability analysis** — set propagation (CORA, Flow\*), cho ra một reachtube  $\{\hat{R}_k\}$ .
- **Certify** → **Signal Temporal Logic (STL)** — robustness định lượng được tính trên cả reachable tube, thay vì chỉ trên một trajectory nominal.
- **Repair** → **CEGIS/CEGAR** — dùng counterexample cụ thể để vừa bác bỏ candidate không an toàn, vừa định hướng sửa.

## Các khối kiến thức có sẵn $\rightarrow$ SHORTSTOP (2/2)



### Đóng góp của SHORTSTOP

Từng công cụ riêng lẻ đều là những khối kiến thức đã hoàn thiện. Đóng góp nằm ở việc **tích hợp chúng thành một runtime shield counterexample-guided duy nhất** cho một generative policy đã đóng băng, kèm safety guarantee cho từng quyết định.

## Part 4 — The P-R-C-S Framework

---

# SHORTSTOP: vòng lặp P-R-C-S

## Tại mỗi decision step $t$

SHORTSTOP giữ nguyên policy  $\pi_\theta$ , lấy một batch candidate chunks, kiểm tra an toàn và chỉ cho phép chunk phù hợp được thực thi.

**P Propose:** Lấy  $K$  candidate từ policy:  $\{a^{(i)}\}_{i=1}^K \sim \pi_\theta(\cdot \mid o_t)$ .

**R Reach:** Lan truyền over-approximation trong short horizon:

$$\hat{\mathcal{R}}_{k+1} = \hat{f}(\hat{\mathcal{R}}_k, a_{t+k}) \oplus \mathcal{B}(\epsilon_k + \bar{w}).$$

**C Certify:** Tính STL robustness-to-go lower bound trên tube:  $\underline{\rho} = \rho(\varphi, \hat{\mathcal{R}})$ .

**S Select/Repair:** Nếu chưa đạt margin, tìm counterexample, sửa trong trust region rồi chọn chunk có task progress tốt nhất.

# P-R-C-S: từ candidate chunk đến action thực thi

## Một decision step tại thời điểm $t$

Policy sinh một **batch nhỏ** candidate chunks; SHORTSTOP chỉ filter các đề xuất này thay vì tìm toàn bộ action space.

### 1. Propose

$$\{a^{(i)}\}_{i=1}^K \sim \pi_{\theta}(\cdot \mid o_t), K = 8$$

### 2. Reach + Certify

Mỗi candidate được:

- (R) propagate thành reachable tube;
- (C) tính STL robustness lower bound.

$$\underline{\rho} \geq \varepsilon \Rightarrow \text{certified admissible}$$

### 3. Select / Repair

Nếu candidate không đạt chứng nhận:

$$\underline{\rho} < \varepsilon$$

SHORTSTOP:

1. tìm một **counterexample** trong reachable tube;
2. dùng nó để xác định hướng repair;
3. sửa candidate trong trust region và quay lại **Reach**.

Nếu có candidate admissible:  $a^* = \arg \max_{a^{(i)} \in C} g(a^{(i)})$ ,  
**chỉ action đầu tiên được execute.**

Nếu tất cả candidate đều fail: chuyển sang **safe fallback controller**, sau đó quan sát lại và re-plan.

## Part 5 — Experimental Setup

---

Ở đâu?

Stress thế nào?

So với ai?

Đo bằng gì?

- **Environments:** từ benchmark manipulation đến controlled reach-avoid setting.
- **Stress tests:** cố tình đưa policy ra khỏi nominal distribution.
- **Baselines:** cùng frozen generative policy, chỉ thay đổi safety mechanism.
- **Metrics:** đánh giá đồng thời safety, task performance, intervention quality và runtime cost.

## Môi trường đánh giá: từ controlled đến realistic

| Testbed         | Stress test cái gì                            | Vai trò               |
|-----------------|-----------------------------------------------|-----------------------|
| LIBERO [56]     | Manipulation dài hạn (long-horizon)           | Benchmark             |
| ManiSkill2 [57] | Độ chính xác khi va chạm nhiều (contact-rich) | Benchmark             |
| RLBench [58]    | Đa dạng tác vụ manipulation                   | Benchmark             |
| Reach-Avoid-2D  | Phân tích an toàn có kiểm soát                | Soundness / prototype |

### Thiết lập chung

Cùng một generative policy đã train bằng imitation learning và đóng băng được bảo vệ bởi mỗi safety method. Không baseline nào được train lại policy riêng.



## Các môi trường đánh giá

### **LIBERO**

Goal / Object / Spatial /  
Long

### **ManiSkill2**

PickCube / StackCube /  
PegInsertion

### **RLBench**

Reach / Close-box /  
Put-in-drawer

### **Reach-Avoid-2D**

Nghiên cứu soundness có  
kiểm soát

Độ thực tế của benchmark



Phân tích có kiểm soát

# Stress test: cố tình rời khỏi nominal distribution

## 1. Object-placement

- Vật cản / mục tiêu bị dịch chuyển
- Tối đa  $\pm 6$  cm
- Đặt vị trí đối nghịch trên nominal path

Sai lệch hình học

## 2. Timing

- Jitter độ trễ điều khiển
- Rớt quan sát
- Lệch thời gian

Sai lệch thời gian

## 3. Visual-shift

- Thay đổi ánh sáng
- Thay đổi texture
- Nhiều camera

Sai lệch tri giác

### Mục đích

Stress test được thiết kế để tăng violation rate khi không có shield (unshielded), và kiểm tra xem shield có can thiệp **chính xác** hay chỉ đơn giản reject tất cả.

## Baselines: sáu hệ thống dưới cùng điều kiện

| Hệ thống           | Cơ chế an toàn                                               |
|--------------------|--------------------------------------------------------------|
| Unshielded         | Generative policy nguyên bản, không có shield                |
| Conf-Thresh        | Reject chunk có độ bất đồng (disagreement) cao               |
| MPC-Filter [33]    | Sửa dự đoán có ràng buộc (constrained predictive correction) |
| CBF-Shield [38,39] | Barrier QP thiết kế tay                                      |
| STL-Monitor [53]   | Giám sát STL trên trajectory nominal                         |
| <b>SHORTSTOP</b>   | Reach + STL + repair theo counterexample                     |

### So sánh có kiểm soát

Tất cả baseline dựa trên model đều dùng chung  $\hat{f}$ , error bound  $\epsilon$ , policy checkpoint đã đóng băng, và fallback controller.

# Metrics I — Safety và chất lượng can thiệp

## Safety

- **Violation rate**

% episode đi vào  $X_u$

Càng thấp càng tốt.

- **Task success**

% episode hoàn thành nhiệm vụ

Càng cao càng tốt.

## Chất lượng can thiệp

- **Shield activation rate**

% quyết định bị reject/repair/fallback

Thường thì càng thấp càng tốt.

- **Intervention precision**

Tỉ lệ can thiệp tương ứng với một vi phạm thật sự sẽ xảy ra.

Càng cao càng tốt.

## Đánh đổi cốt lõi

Một shield tốt cần giảm violation **mà không đơn giản là reject tất cả.**

## Metrics II — Chi phí runtime và giá trị can thiệp

### Latency

ms tăng thêm (trung vị) / quyết định

Đo overhead runtime.

### Conservatism cost

$\Delta \text{Success}$

trên các episode không có sự  
kiện an toàn thật sự.

Đo mức suy giảm hiệu năng  
không cần thiết.

### Recovery rate

nhiệm vụ thành công sau can thiệp  
trường hợp chunk bị reject

Đo xem repair có phục hồi  
được hiệu năng nhiệm vụ hay  
không.

### Vì sao các metric này quan trọng

Chỉ an toàn thôi là chưa đủ:

An toàn  $\nrightarrow$  Hữu ích

Một shield thực tế cần **an toàn, chọn lọc, phục hồi được, và nhanh.**

## Part 6 — Guarantees & Challenges

---

## Vì sao khó để có được các guarantee này?

- **Epsilon calibration:** calibrate  $\varepsilon$  đúng theo phân phối thật, không phải một hằng số cố định.
- **Model error dưới distribution shift:**  $\varepsilon$  sai dưới shift  $\Rightarrow$  certificate sập âm thầm mà không cảnh báo.
- **Safe fallback:** xây  $X_s, \pi_{fb}$  hợp lệ — không tự động đúng, phải thiết kế riêng.
- **Safety vs. liveness:** 22% reject không recover; certify *batch* chunk từ hộp đen.
- **Real-time constraint:** giải bài toán đối nghịch đủ nhanh và đúng, trong ngân sách  $<10\text{ms}$  cạnh GPU policy.

*3 lát cắt khó xuyên suốt: khái niệm (đúng ranh giới tractable/intractable), toán học (đúng dẫn dưới bất định mà không biết cấu trúc  $\pi_\theta$ ), hệ thống ( $<10\text{ms}$  cạnh GPU policy).*

## Assumptions: điều kiện để guarantee đứng vững

- Mỗi challenge ở trên cần một giả định tương ứng để lý thuyết còn đứng vững:
- **Assumption 1:** reachable-set sound — reachtube phải chứa quỹ đạo thật (đối phó với model error dưới distribution shift).
- **Assumption 2:** STL bound sound —  $\rho(\phi, \hat{R})$  phải là cận dưới đúng của robustness thật (đối phó với sai số calibrate  $\varepsilon$ ).
- **Assumption 3:** tồn tại safe fallback  $(\pi_{fb}, X_s)$  — luôn có đường lui an toàn (chính là challenge “safe fallback” ở trên).
- Mỗi assumption cũng bảo vệ đúng một mắt xích trong vòng P-R-C-S: (1)→Reach, (2)→Certify, (3)→Select/Repair khi rỗng.



## Từ Bounded-time đến All-time safety

- **Theorem 1** (bounded-time safety): với 3 assumption trên, chunk được ShortStop chấp nhận không vi phạm  $X_u$  trong  $H$  bước tới.
- **Corollary 1** (all-time safety): quy nạp theo  $t$  dựa trên  $X_s$  bất biến — an toàn từng bước cộng dồn thành an toàn suốt quỹ đạo.

## Conservatism vs. Horizon

- **Proposition 1** (Grönwall-type, Eq. 6–7):  $r_k \leq L^k r_0 + (\varepsilon_{\max} + \bar{w}) \frac{L^k - 1}{L - 1}$ ,  
 $\alpha(H) \leq \alpha^* + \beta(\epsilon + r_H)$ .
- Đây chính là cơ sở toán học cho trực giác “ngắn hạn vừa đủ vừa cần” ở Part 3:  $L^k$  tăng theo horizon  $\Rightarrow$  tồn tại  $H^*$  tối ưu (thực nghiệm  $H^* = 8$ ).

## Track A & Track B — Kế hoạch song song

---

## Track A & Track B: hai kế hoạch chạy song song

- **Track A (Lý thuyết):** Sec. I–IV — bài toán, related work, method, các định lý an toàn, chia theo 7 nhóm chủ đề.
- **Track B (Thực nghiệm):** Sec. V–VIII — dựng pipeline trên Reach-Avoid-2D với policy Gaussian giả lập trước (stage 0–5), chỉ đưa generative policy thật vào từ stage 6a, rồi mới scale lên 3 benchmark thật (stage 7a–7c).
- **Vì sao chưa dùng generative policy ngay:** shield logic (reach/STL/CE-search/repair/baselines) cần debug nhiều vòng — Gaussian rẻ và nhanh hơn; policy thật chỉ cần thiết khi kiểm tra đúng giả định “black-box proposer”.
- 3 bảng dưới đây ghép mỗi stage Track B với nhóm lý thuyết Track A tương ứng, kèm status theo dõi.

## Track A & Track B: lộ trình song song (1/3)

| Track A                                                                                                                     | Status         | Track B                                                                                                         | Status                   |
|-----------------------------------------------------------------------------------------------------------------------------|----------------|-----------------------------------------------------------------------------------------------------------------|--------------------------|
| <b>Stage 0</b> — Reachability cơ bản (interval/zonotope, Minkowski sum, Lipschitz) + Shielding/Safe RL context (đọc nhanh). | Đang thực hiện | Setup — env Reach-Avoid-2D, interval arithmetic, cvxpy; policy Gaussian giả lập chỉ để dựng khung.              | Đã xong                  |
| <b>Stage 1</b> — Reachtube & Assumption 1 (sound over-approximation).                                                       | Chưa bắt đầu   | Reach-only shield — reachtube, binary reject (Gaussian; violation $\approx 3.8\%$ , success $\approx 64.1\%$ ). | Đã xong                  |
| <b>Stage 2</b> — STL syntax & robustness-to-go, Assumption 2.                                                               | Chưa bắt đầu   | STL-to-go — thay reject nhị phân bằng robustness lower bound (Gaussian; kỳ vọng $\approx 2.9\%$ violation).     | Chưa bắt đầu (tiếp theo) |
| <b>Stage 3</b> — CEGIS: tìm counterexample (adversarial search).                                                            | Chưa bắt đầu   | CE search — projected gradient/vertex sampling trên box 2D (Gaussian; kỳ vọng $\approx 1.7\%$ violation).       | Chưa bắt đầu             |

## Track A & Track B: lộ trình song song (2/3)

| Track A                                                                                       | Status       | Track B                                                                                                       | Status       |  |
|-----------------------------------------------------------------------------------------------|--------------|---------------------------------------------------------------------------------------------------------------|--------------|--|
| <b>Stage 4</b> — CEGIS: sửa theo counterexample (trust-region projection).                    | Chưa bắt đầu | Repair — Algorithm 1 đầy đủ (Gaussian; kỳ vọng $\approx 1.2\%$ violation).                                    | Chưa bắt đầu |  |
| <b>Stage 5</b> — MPC/CBF & fallback, Assumption 3 (safe fallback tồn tại).                    | Chưa bắt đầu | Baselines — MPC-Filter, CBF-Shield, STL-Monitor, Conf-Thresh (vẫn trên Gaussian).                             | Chưa bắt đầu |  |
| <b>Stage 6a</b> — Generative policy, bản nhỏ (Diffusion Policy/flow matching).                | Chưa bắt đầu | 2D policy thật — train một diffusion/flow policy nhỏ trên Reach-Avoid-2D, hoặc import policy có sẵn.          | Chưa bắt đầu |  |
| <b>Stage 6b</b> — Safety guarantee (bounded-time $\rightarrow$ all-time, Grönwall trade-off). | Chưa bắt đầu | Metrics + stress test — 7 metric, horizon sweep, bootstrap, <b>chạy lại với policy 2D thật thay Gaussian.</b> | Chưa bắt đầu |  |

## Track A & Track B: lộ trình song song (3/3)

| Track A                                                            | Status       | Track B                                                          | Status                          |
|--------------------------------------------------------------------|--------------|------------------------------------------------------------------|---------------------------------|
| <b>Stage 7a</b> — Generative policy, scale lên (cùng nhóm với 6a). | Chưa bắt đầu | <b>LIBERO</b> — swap sang env thật, long-horizon manipulation.   | Chưa bắt đầu (tùy resource GPU) |
| <b>Stage 7b</b> — Generative policy, scale lên (cùng nhóm với 6a). | Chưa bắt đầu | <b>ManiSkill2</b> — swap sang env thật, contact-rich precision.  | Chưa bắt đầu (tùy resource GPU) |
| <b>Stage 7c</b> — Generative policy, scale lên (cùng nhóm với 6a). | Chưa bắt đầu | <b>RLBench</b> — swap sang env thật, diverse manipulation tasks. | Chưa bắt đầu (tùy resource GPU) |

*Số liệu LIBERO/ManiSkill2/RLBench trong Part 5 hiện là projection, không phải full simulator run — stage 7a–7c chính là phần lấp khoảng trống đó.*

## Tổng kết

---



- **Insight:** verify hậu quả ngắn hạn của 1 chunk, không verify cả policy.
- **Cơ chế:** reachtube sound + STL robustness-to-go + counterexample-guided reject/repair.
- **Bảo đảm:** bounded-time  $\rightarrow$  all-time (recurrent fallback)  $\rightarrow$  conservatism–horizon trade-off.
- **Kết quả (của paper):** violation 18.7%  $\rightarrow$  1.2%, giữ 92% success, +8.9ms, precision 0.94.
- **Vị trí:** điểm giao mới giữa 3 dòng nghiên cứu (post-hoc filter, reachability+STL, CEGIS) — chưa ai làm cả ba cùng lúc cho generative robot policy.
- **Hướng tái lập và nghiên cứu tiếp theo:** Track A (lý thuyết, Module 1–4) + Track B (thực nghiệm, Phase 0–6) chạy song song — Phase 0–1 đã xong, Phase 2 là bước tiếp theo.